



DailyPlanet Technical Design Document

Ver 1.0

Contents

DailyPlanet Technical Design Document	4
1. Executive Summary	4
2. High-Level Architecture	4
3. Technology Stack	6
4. System Components	6
4.1 Frontend Components	6
4.2 API Layer	6
4.3 Backend Components	7
5. Request Flow	7
5.1 News Feed Flow	7
5.2 Search Flow	8
6. Provider Architecture	8
6.1 Provider Interface	8
6.2 Provider Registry	8
6.3 Provider Implementations	9
6.4 Provider Execution	9
6.5 Extensibility	9
7. Filtering System	9
7.1 Category Filtering	10
7.2 Country Filtering	10
7.3 Date Filtering	10
7.4 Capability-Based Provider Selection	10
8. Pagination Design	11
8.1 Problem Statement	11
8.2 Composite Cursor Model	11
8.3 Cursor Encoding	11
8.4 Pagination Workflow	12
8.5 Benefits	12
9. Error Handling & Fault Tolerance	12
9.1 Request Validation	12
9.2 Provider Failures	12
9.3 Environment Validation	13
9.4 Rate Limiting	13
9.5 Fault Tolerance Strategy	13
10. Performance Optimizations	13
10.1 Parallel Provider Execution	14
10.2 Server-Side Caching	14
10.3 Infinite Scrolling	14
10.4 Aggregation and Deduplication	14
10.5 Rate Limiting	14
11. API Specification	14
11.1 News Endpoint	15
11.2 Search Endpoint	15
11.3 Unified Article Schema	16

11.4	Pagination	16
11.5	Error Responses	16
12.	Testing Strategy	17
12.1	Unit Testing	17
12.2	Integration Testing	17
12.3	External API Mocking	17
12.4	End-to-End Testing	18
12.5	Test Coverage Goals	18
13.	Scalability Considerations	18
13.1	Provider Scalability	18
13.2	Feature Scalability	19
13.3	Traffic Scalability	19
13.4	Data Source Scalability	19
13.5	Future Scaling Opportunities	19
14.	Future Improvements	19

DailyPlanet Technical Design Document

1. Executive Summary

DailyPlanet is a news aggregation platform built using Next.js 16, React 19, and TypeScript. The system aggregates articles from multiple news providers, including The Guardian, NewsData.io, and Currents API, and presents them through a unified interface for browsing and search.

The application is designed around a provider-based architecture. Each provider implements a common interface, allowing the system to normalize different API responses into a shared article schema. An orchestration layer coordinates requests across providers, aggregates results, removes duplicate articles, and returns a chronologically sorted feed to the client.

The platform supports category-based browsing, keyword search, country filtering, date-range filtering, and infinite scrolling. To provide a consistent user experience despite differing provider pagination mechanisms, the system employs a composite cursor pagination model that combines provider-specific pagination state into a single encoded token.

Performance and reliability are improved through parallel provider requests, server-side response caching, Redis-backed rate limiting, and fault-tolerant request handling. The architecture is designed to be extensible, allowing additional news providers to be integrated with minimal changes to the core system.

This document describes the architecture, component interactions, design decisions, data flow, and scalability considerations of the DailyPlanet platform.

2. High-Level Architecture

DailyPlanet uses a layered architecture to aggregate news from multiple external providers through a unified interface. User requests are handled by Next.js API routes, which delegate processing to an orchestration layer responsible for provider selection and request execution.

Provider responses are normalized into a common article schema before being aggregated, deduplicated, sorted, and returned to the client. This design isolates provider-specific logic while providing a consistent experience for search, filtering, and pagination

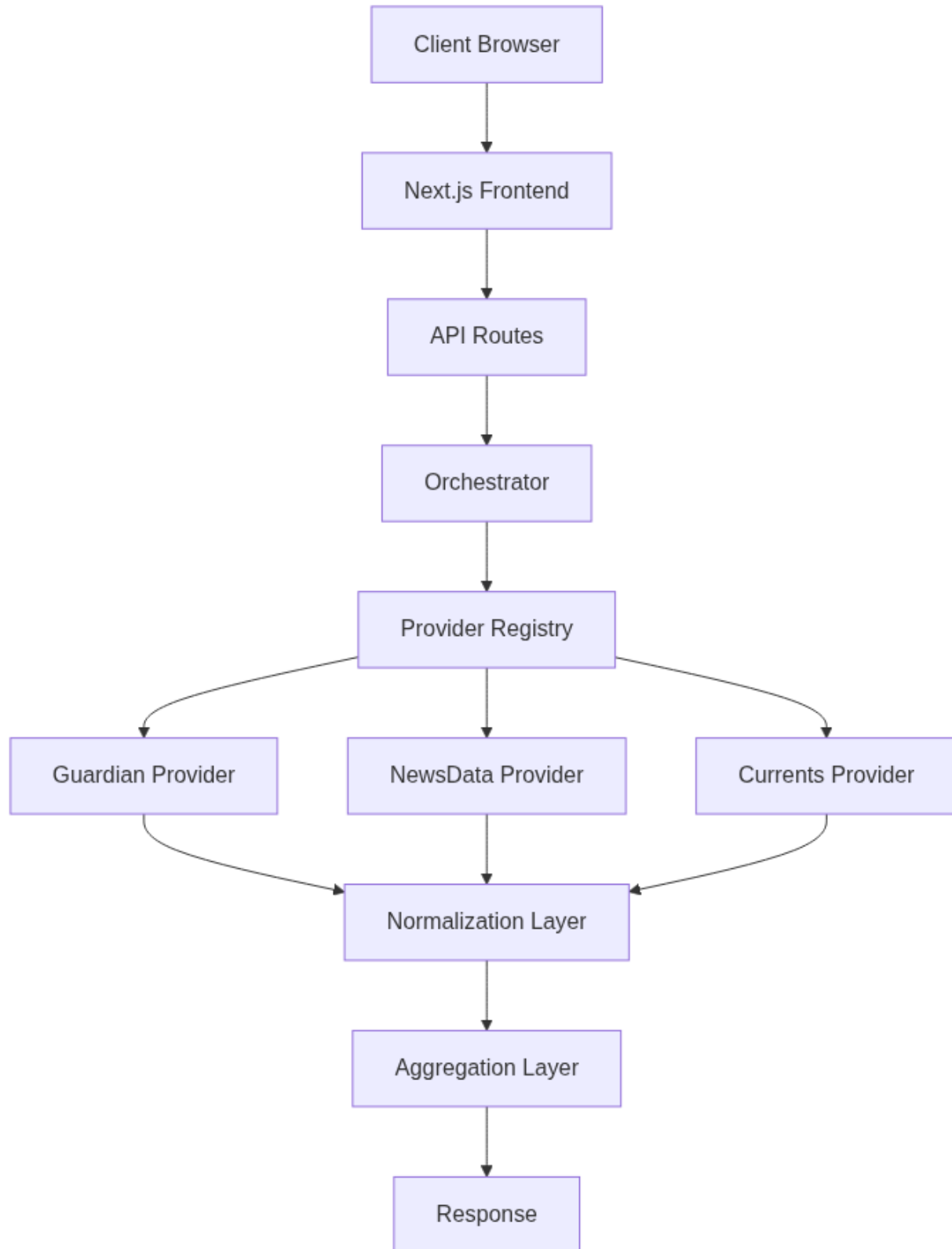


Figure 1: High-Level Architecture

3. Technology Stack

Technology	Purpose
Next.js 16	Frontend framework and API routes
React 19	User interface development
TypeScript	Type safety and maintainability
Upstash Redis	Distributed rate limiting
Guardian API	News provider
NewsData.io	News provider
Currents API	News provider
Vercel	Application hosting and deployment

4. System Components

This section describes the major software components that make up the DailyPlanet platform and their responsibilities.

4.1 Frontend Components

The frontend is built using Next.js and React and is responsible for displaying aggregated news content and managing user interactions.

Component	Responsibility
NewsFeed	Displays articles and manages infinite scrolling
NewsCard	Renders individual article content and metadata
GlobalSearchBar	Handles keyword-based searching
CountryFilter	Applies country-based filtering
CatTabs	Manages category selection and navigation

4.2 API Layer

The API layer exposes server-side endpoints used by the frontend.

`/api/news` Responsible for:

- Retrieving aggregated headlines
- Applying filters
- Managing pagination
- Enforcing rate limits

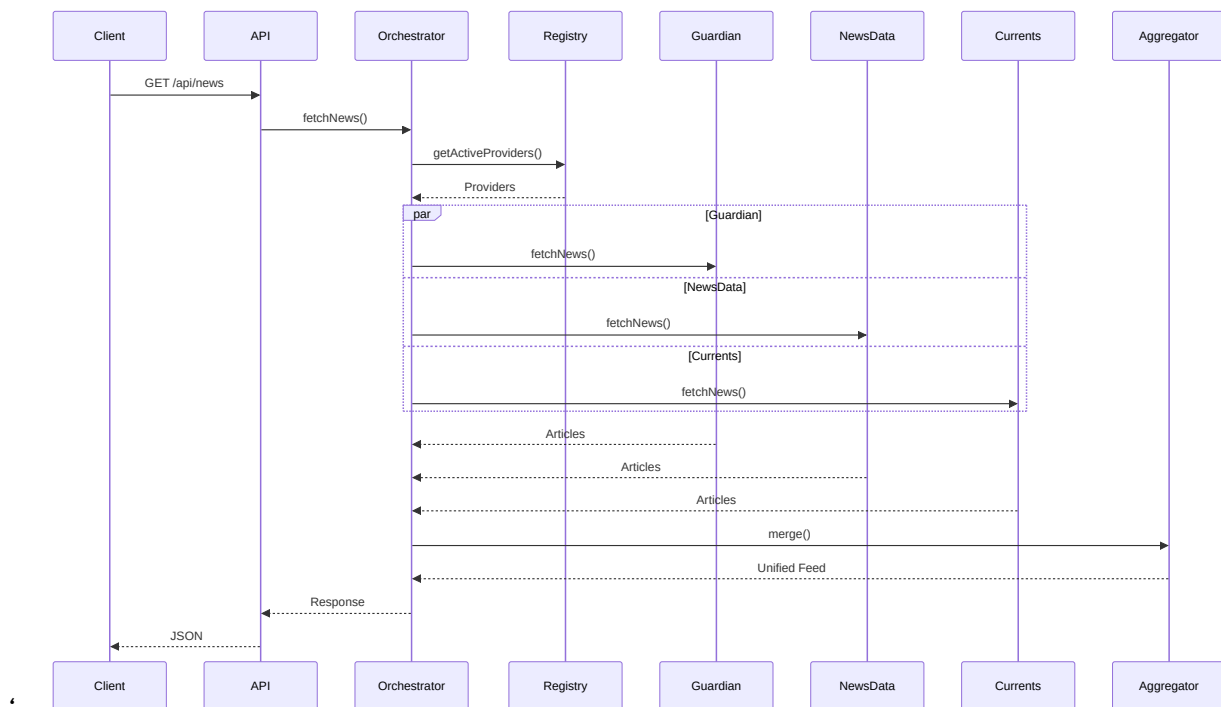
`/api/search` Responsible for:

- Executing cross-provider searches
- Returning normalized search results
- Managing pagination

4.3 Backend Components

Component	Location	Responsibility
Orchestrator	<code>src/lib/news/orchestrator.ts</code>	Coordinates provider execution
Registry	<code>src/lib/news/registry.ts</code>	Selects compatible providers
Normalizer	<code>src/lib/normalize.ts</code>	Standardizes provider responses
Aggregator	<code>src/lib/aggregate.ts</code>	Merges and deduplicates articles
Pagination	<code>src/lib/news/pagination.ts</code>	Generates composite cursors
Rate Limiter	<code>src/lib/rateLimit.ts</code>	Enforces API limits

5. Request Flow



5.1 News Feed Flow

1. The client sends a request to `/api/news` with optional filtering and pagination parameters.
2. The API layer validates the request and applies rate limiting.
3. The orchestration layer selects compatible providers through the provider registry.

4. Active providers are queried concurrently.
5. Responses are normalized, aggregated, deduplicated, and sorted.
6. A composite pagination cursor is generated.
7. The aggregated response is returned to the client.

5.2 Search Flow

The search workflow follows the same execution pipeline as the news feed workflow, but includes a user-provided search query and executes provider search operations instead of headline retrieval.

6. Provider Architecture

DailyPlanet uses a provider-based architecture to abstract interactions with external news services. Each news source is implemented as a provider that conforms to a common interface, allowing the rest of the system to interact with all providers in a consistent manner regardless of their underlying API implementations.

This approach isolates provider-specific logic from the orchestration and aggregation layers, simplifying maintenance and enabling new providers to be integrated with minimal changes to the core system.

6.1 Provider Interface

All providers implement a shared interface that defines the operations required by the orchestration layer.

The interface standardizes:

- News retrieval
- Search operations
- Pagination handling
- Provider capability metadata
- Response normalization

By enforcing a common contract, the orchestration layer can interact with all providers without requiring provider-specific logic.

6.2 Provider Registry

The provider registry maintains a list of all available providers and is responsible for provider selection.

Before a request is executed, the registry evaluates provider capabilities and excludes providers that cannot satisfy the requested filters.

Current provider capabilities include:

- Country filtering support
- Date-range filtering support

This ensures that only compatible providers participate in request execution.

6.3 Provider Implementations

The system currently supports three providers:

1. Guardian
2. NewsData.io
3. Currents

Each provider is responsible for:

- Constructing provider-specific API requests
- Mapping categories to provider-specific values
- Handling pagination
- Normalizing responses into the common article schema

6.4 Provider Execution

Once active providers have been selected, the orchestration layer executes provider requests concurrently using JavaScript's `Promise.all()` mechanism.

```
const results = await Promise.all(
  providers.map(async (provider) => {
    try {
      return await provider.fetch(request)
    } catch {
      return {
        articles: [],
        nextCursor: undefined
      }
    }
  })
)
```

6.5 Extensibility

The provider architecture was designed to support future expansion.

To integrate a new news source, a developer must:

1. Implement the provider interface.
2. Register the provider in the provider registry.
3. Configure category mappings and provider capabilities.

No modifications to the orchestration or aggregation layers are required, allowing the platform to scale horizontally as additional providers are introduced.

7. Filtering System

DailyPlanet supports category, country, and date-range filtering across multiple news providers. Since providers expose different filtering capabilities, the system uses capability-based provider selection to ensure only compatible providers participate in request execution.

Before a request is processed, the provider registry evaluates the requested filters and excludes providers that cannot satisfy them. This prevents unsupported filters from producing inconsistent or inaccurate results.

7.1 Category Filtering

Category filtering is supported by all providers.

User-facing categories are mapped to provider-specific category values through a centralized category mapping layer. This abstraction allows a single category selection to be translated into the appropriate format required by each external API.

7.2 Country Filtering

Country filtering is supported by NewsData and Currents providers.

When a country filter is specified, providers that do not support country-based filtering are automatically excluded from execution.

This ensures that all returned articles satisfy the requested geographic constraints.

7.3 Date Filtering

Date-range filtering is supported by Guardian and Currents providers.

When a date range is supplied, the provider registry excludes providers that cannot perform date-based filtering.

This allows the system to return results that accurately reflect the requested time window without requiring additional client-side filtering.

7.4 Capability-Based Provider Selection

Provider selection is performed dynamically using capability metadata exposed by each provider.

```
if (opts.country && !provider.supportsCountryFilter)
  return false

if (opts.startDate && opts.endDate && !provider.supportsDateFilter)
  return false
```

The registry evaluates provider capabilities before request execution and automatically excludes providers that cannot satisfy the requested filters.

Provider	Category Filter	Country Filter	Date Filter
Guardian	✓	x	✓
NewsData	✓	✓	x
Currents	✓	✓	✓

This approach allows filtering behavior to remain consistent while supporting providers with different feature sets.

8. Pagination Design

DailyPlanet aggregates content from multiple news providers, each of which exposes a different pagination mechanism. While Guardian and Currents use page-based pagination, NewsData uses cursor-based pagination.

This creates a challenge when presenting a single unified feed to the client, as the application must maintain pagination state for multiple providers simultaneously.

8.1 Problem Statement

A traditional pagination model assumes that all data originates from a single source with a single pagination strategy.

In DailyPlanet, each provider maintains its own pagination state:

- Guardian uses page numbers
- Currents uses page numbers
- NewsData uses cursor tokens

As a result, the client cannot directly interact with provider-specific pagination mechanisms.

8.2 Composite Cursor Model

To provide a consistent pagination experience, DailyPlanet uses a composite cursor model.

The pagination state of all active providers is combined into a single object:

```
{  
  "guardianPage": 2,  
  "newsDataCursor": "abc123",  
  "currentsPage": 3  
}
```

This object represents the complete pagination state required to retrieve the next batch of results.

8.3 Cursor Encoding

The composite pagination state is encoded and returned to the client as a single pagination token.

The client treats this token as an opaque value and includes it in subsequent requests when requesting additional results.

This approach hides provider-specific implementation details from the client while maintaining support for heterogeneous pagination strategies.

8.4 Pagination Workflow

1. Providers return articles and updated pagination state.
2. The orchestration layer combines provider pagination information.
3. The pagination layer generates a composite cursor.
4. The cursor is encoded and returned as the `nextPage` value.
5. The client supplies the cursor in subsequent requests.
6. The pagination layer decodes the cursor and restores provider-specific state.

8.5 Benefits

The composite cursor model provides several advantages:

- Unified pagination across multiple providers
- Provider-specific pagination complexity remains hidden from clients
- Support for heterogeneous pagination mechanisms
- Simplified frontend implementation
- Extensibility for future providers

9. Error Handling & Fault Tolerance

DailyPlanet incorporates multiple error handling mechanisms to ensure reliability when interacting with external services and processing client requests.

9.1 Request Validation

Incoming requests are validated before processing begins.

Validation includes:

- Required parameter checks
- Date range validation
- Category validation
- Pagination cursor validation

Invalid requests are rejected with an appropriate error response before any provider requests are executed.

9.2 Provider Failures

The platform depends on multiple third-party news providers, each of which may experience downtime, rate limits, or unexpected API changes.

Provider requests are isolated from one another, allowing the system to continue operating even if an individual provider fails.

If a provider encounters an error, the failure is logged and remaining providers continue processing the request.

This approach prevents a single provider outage from making the entire service unavailable.

9.3 Environment Validation

During application startup, the provider registry validates that all required API credentials are present.

```
const missingKeys: string[] = []

if (!process.env.GUARDIANAPI_KEY)
  missingKeys.push("GUARDIANAPI_KEY")

if (!process.env.NEWSDATA_API_KEY)
  missingKeys.push("NEWSDATA_API_KEY")

if (!process.env.CURRENTNEWS_API_KEY)
  missingKeys.push("CURRENTNEWS_API_KEY")
```

If required configuration values are missing, the application fails fast and reports the missing environment variables.

9.4 Rate Limiting

To prevent abuse and protect external API quotas, server-side rate limits are enforced using Upstash Redis.

Requests exceeding configured limits receive an error response and are prevented from executing provider requests.

This protects both infrastructure resources and third-party API quotas.

9.5 Fault Tolerance Strategy

The system is designed to degrade gracefully when failures occur.

Key fault-tolerance measures include:

- Provider isolation
- Independent provider execution
- Partial result delivery
- Configuration validation
- Rate limiting

These mechanisms ensure that the platform remains operational even when external dependencies become unavailable.

10. Performance Optimizations

DailyPlanet incorporates several optimizations to reduce response times, minimize external API usage, and improve scalability.

10.1 Parallel Provider Execution

News providers are queried concurrently rather than sequentially.

By executing provider requests in parallel, the total response time is determined by the slowest provider rather than the combined latency of all providers.

This significantly reduces the time required to aggregate content from multiple sources.

10.2 Server-Side Caching

External API requests are cached using Next.js fetch revalidation.

Cached responses can be served without contacting the underlying provider, reducing latency and decreasing the number of external API requests.

Caching also helps preserve limited API quotas provided by third-party services.

10.3 Infinite Scrolling

The frontend implements infinite scrolling to load articles incrementally.

Instead of retrieving a large number of articles during the initial request, content is loaded in smaller batches as the user navigates through the feed.

This reduces initial page load times and improves perceived responsiveness.

10.4 Aggregation and Deduplication

Provider responses are normalized and aggregated on the server before being returned to the client.

Duplicate articles are removed during aggregation, reducing unnecessary data transfer and preventing repeated content from appearing in the feed.

10.5 Rate Limiting

Server-side rate limiting is enforced using Upstash Redis.

Rate limiting protects external API quotas and prevents excessive request volumes from impacting application performance.

By controlling traffic at the API layer, the system can maintain predictable resource utilization under varying load conditions.

11. API Specification

DailyPlanet exposes two primary API endpoints that provide access to aggregated news content and search functionality.

11.1 News Endpoint

Route

GET /api/news

Purpose

Returns aggregated news articles from all compatible providers.

Supported Parameters

Parameter	Description
category	Filter articles by category
country	Filter articles by country
startDate	Start of date range
endDate	End of date range
page	Composite pagination cursor

Response Structure

```
{
  "success": true,
  "articles": [...],
  "nextPage": "encodedCursor"
}
```

11.2 Search Endpoint

Route

GET /api/search

Purpose

Performs keyword-based searches across all compatible providers.

Supported Parameters

Parameter	Description
query	Search term
category	Filter results by category
country	Filter results by country
startDate	Start of date range
endDate	End of date range
page	Composite pagination cursor

Response Structure

```
{
  "success": true,
  "articles": [...],
  "nextPage": "encodedCursor"
}
```

11.3 Unified Article Schema

All provider responses are normalized into a common schema before being returned to clients.

```
{
  title: string
  url: string
  category: string[]
  source: string
  publishedAt: string
  content: string
  byline?: string
  thumbnail?: string
}
```

This abstraction allows client applications to consume news content without requiring knowledge of provider-specific response formats.

11.4 Pagination

Both endpoints support cursor-based pagination through the `page` parameter.

The cursor contains encoded provider-specific pagination state and is generated by the pagination layer.

Clients should treat the cursor as an opaque value and pass it unchanged in subsequent requests.

11.5 Error Responses

Error responses follow a consistent structure:

```
{
  "success": false,
  "error": "Error message"
}
```

Errors may be returned for:

- Invalid request parameters
- Missing search terms

- Rate limit violations
- Internal server errors

12. Testing Strategy

DailyPlanet is designed to support testing at multiple levels, including unit testing, integration testing, and end-to-end testing. This layered approach ensures that individual components, system interactions, and user workflows can be validated independently.

12.1 Unit Testing

Unit tests focus on validating isolated business logic without requiring external dependencies.

Key areas for unit testing include:

- Article normalization
- Category mapping
- Aggregation and deduplication
- Cursor encoding and decoding
- Provider selection logic

These tests ensure that core functionality behaves correctly under a variety of inputs and edge cases.

12.2 Integration Testing

Integration tests verify interactions between internal components and API routes.

Key integration test scenarios include:

- News retrieval through `/api/news`
- Search functionality through `/api/search`
- Pagination handling
- Filter application
- Rate limiting behavior

Integration testing ensures that requests are processed correctly throughout the application pipeline.

12.3 External API Mocking

Third-party news providers should be mocked during automated testing to avoid dependency on external services.

Mocking provides several benefits:

- Consistent test results
- Faster execution
- Reduced API quota consumption
- Isolation from provider downtime

Provider responses can be simulated using representative sample payloads obtained from Guardian, NewsData, and Currents APIs.

12.4 End-to-End Testing

End-to-end testing validates complete user workflows through the frontend interface.

Example scenarios include:

- Browsing news articles
- Performing searches
- Applying category filters
- Applying country filters
- Applying date-range filters
- Loading additional content through infinite scrolling

These tests verify that the application behaves correctly from a user's perspective.

12.5 Test Coverage Goals

The primary testing focus should be placed on the system's core business logic and integration points.

High-priority areas include:

- Provider orchestration
- Capability-based provider selection
- Pagination state management
- Aggregation and deduplication
- API endpoint behavior

Testing these components provides confidence in the correctness and reliability of the platform.

13. Scalability Considerations

DailyPlanet was designed with scalability and maintainability as primary architectural goals. The system's modular structure allows individual components to evolve independently as requirements grow.

13.1 Provider Scalability

The provider-based architecture allows additional news sources to be integrated without modifying the orchestration or aggregation layers.

To add a new provider, a developer must:

1. Implement the provider interface.
2. Register the provider in the provider registry.
3. Configure category mappings and capability metadata.

This approach enables the platform to scale horizontally as additional content sources are introduced.

13.2 Feature Scalability

The capability-based provider selection mechanism allows new filtering features to be introduced incrementally.

Providers can advertise support for new capabilities without requiring changes to existing provider implementations.

This reduces coupling between providers and core application logic.

13.3 Traffic Scalability

The application is built on Next.js server-side infrastructure and incorporates caching and rate limiting to reduce load on both internal services and external providers.

As traffic increases, additional caching strategies and distributed deployments can be introduced without significant architectural changes.

13.4 Data Source Scalability

Because all provider responses are normalized into a common article schema, the aggregation layer remains independent of provider-specific response formats.

This ensures that increasing the number of providers does not significantly increase system complexity.

13.5 Future Scaling Opportunities

Potential future enhancements include:

- Additional news providers
- Distributed caching layers
- Search indexing for faster search performance
- Provider health monitoring
- Personalized news recommendations
- Analytics dashboards

14. Future Improvements

Several enhancements can be made to improve functionality, scalability, and user experience.

- Integration of additional news providers
- Full-text search indexing
- Personalized article recommendations
- Provider health monitoring
- Analytics and reporting dashboards
- Advanced caching strategies
- User accounts and saved article functionality

These improvements can be implemented without significant architectural changes due to the modular provider-based design of the platform.